

**LAPORAN TUGAS BESAR**  
**MILESTONE IV – INTERMEDIATE CODE & INTERPRETER**  
**IF2224 TEORI BAHASA FORMAL DAN AUTOMATA**  
**ARION COMPILER**



Disusun oleh:

Kelompok 11

**“noooRegex” - MMD**

|                            |          |
|----------------------------|----------|
| Kloce Paul William Saragih | 13524040 |
| Suryani Mulia Utami        | 13524042 |
| Josh Reinhart Zidik        | 13524048 |
| Syaqina Octavia Rizha      | 13524088 |

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**  
**PROGRAM STUDI TEKNIK INFORMATIKA**  
**INSTITUT TEKNOLOGI BANDUNG**  
**2025/2026**

## DAFTAR ISI

|                                                |           |
|------------------------------------------------|-----------|
| <b>BAB 1.....</b>                              | <b>1</b>  |
| <b>PENDAHULUAN.....</b>                        | <b>1</b>  |
| <b>BAB 2.....</b>                              | <b>2</b>  |
| <b>DASAR TEORI.....</b>                        | <b>2</b>  |
| 2.1. Sintesis dan Eksekusi.....                | 2         |
| 2.2. Intermediate Code Generation.....         | 2         |
| 2.3. Interpreter.....                          | 4         |
| <b>BAB 3.....</b>                              | <b>6</b>  |
| <b>PERANCANGAN.....</b>                        | <b>6</b>  |
| 3.1. Desain Intermediate Code Generation.....  | 6         |
| 3.1.1. Format Instruksi Bytecode.....          | 6         |
| 3.1.2. Pembangkitan Kode Program.....          | 7         |
| 3.1.3. Pembangkitan Ekspresi.....              | 7         |
| 3.1.4. Pembangkitan Statement.....             | 8         |
| 3.1.5. Manajemen Lexical Level dan Alamat..... | 9         |
| 3.2. Desain Interpreter.....                   | 9         |
| 3.2.1. Siklus Eksekusi.....                    | 10        |
| 3.2.2. Siklus Instruksi.....                   | 10        |
| 3.2.3. Penanganan Kesalahan Runtime.....       | 11        |
| 3.3. Teknologi.....                            | 12        |
| 3.4. Struktur Program.....                     | 12        |
| 3.5. Fungsi Utama.....                         | 13        |
| 3.6. Alur Kerja Program.....                   | 17        |
| 3.7. Pendekatan Implementasi.....              | 20        |
| <b>BAB 4.....</b>                              | <b>22</b> |
| <b>HASIL IMPLEMENTASI.....</b>                 | <b>22</b> |
| <b>BAB 5.....</b>                              | <b>29</b> |
| <b>EKSPERIMEN.....</b>                         | <b>29</b> |
| <b>BAB 6.....</b>                              | <b>46</b> |
| <b>KESIMPULAN DAN SARAN.....</b>               | <b>46</b> |
| 6.1. Kesimpulan.....                           | 46        |
| 6.2. Saran.....                                | 46        |
| <b>REFERENSI.....</b>                          | <b>47</b> |

## DAFTAR GAMBAR

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Gambar 1.1 Skema Kerja Interpreter.....                                       | 1  |
| Gambar 2.1.1 Proses Interpreted Language dari Tokenisasi hingga Eksekusi..... | 2  |
| Gambar 2.3.1.1. Mekanisme Operasi Stack.....                                  | 5  |
| Gambar 3.6.1 Alur Kerja Program.....                                          | 17 |

## DAFTAR TABEL

|                                                        |    |
|--------------------------------------------------------|----|
| Tabel 3.1.1.1 Opcode yang Didefinisikan.....           | 6  |
| Tabel 3.2.2.1 Instruksi yang Didukung Interpreter..... | 10 |
| Tabel 3.2.1 Teknologi yang Digunakan.....              | 12 |
| Tabel 3.5.1 Desain Fungsi Utama.....                   | 14 |
| Tabel 5.1 Kasus Uji 1.....                             | 29 |
| Tabel 5.2 Kasus Uji 2.....                             | 29 |
| Tabel 5.3 Kasus Uji 3.....                             | 34 |
| Tabel 5.4 Kasus Uji 4.....                             | 35 |
| Tabel 5.5 Kasus Uji 5.....                             | 38 |

# BAB 1

## PENDAHULUAN

Bahasa pemrograman digunakan untuk menuliskan instruksi agar komputer dapat menjalankan suatu program tanpa harus menggunakan bahasa mesin secara langsung. Komputer hanya memahami bahasa mesin dalam bentuk biner (0 dan 1), sehingga bahasa pemrograman dikembangkan agar lebih mudah dipahami manusia. Secara umum, bahasa pemrograman dibedakan menjadi low-level yang dekat dengan bahasa mesin dan high-level yang lebih mendekati bahasa manusia. Semakin tinggi tingkat bahasanya, semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.

Salah satu cara untuk menerjemahkan bahasa pemrograman ke dalam kode mesin adalah dengan menggunakan interpreter. Interpreter bekerja dengan membaca, menerjemahkan, dan mengeksekusi kode sumber secara baris demi baris. Pendekatan ini memudahkan dalam menemukan kesalahan karena error dapat langsung diketahui pada baris tertentu. Berbeda dengan compiler yang menerjemahkan seluruh source code menjadi kode mesin (misalnya dalam bentuk object file) sebelum program dijalankan.

Meskipun berbeda dalam cara eksekusi, keduanya tetap melalui tahapan analisis yang serupa. Proses dimulai dari Lexical Analysis yang memecah kode sumber menjadi token-token kecil seperti kata kunci, variabel, dan operator. Selanjutnya, Syntax Analysis yang memeriksa apakah susunan token tersebut sudah sesuai dengan aturan tata bahasa (*grammar*) yang berlaku. Setelah itu, Semantic Analysis memastikan bahwa makna dari kode tersebut benar. Tahap terakhir, Intermediate Code Generation yang mengubah kode ke bentuk perantara yang lebih mudah diproses sebelum akhirnya dieksekusi atau diterjemahkan lebih lanjut menjadi kode mesin.



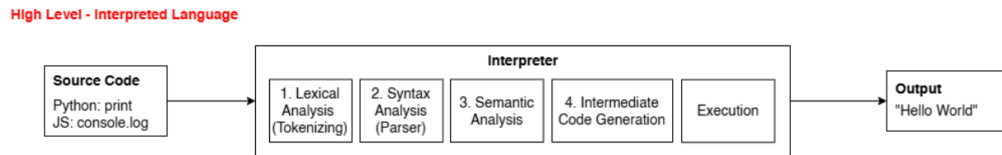
*Gambar 1.1 Skema Kerja Interpreter*

*Sumber: GeeksForGeeks*

## BAB 2

### DASAR TEORI

#### 2.1. Sintesis dan Eksekusi



Gambar 2.1.1 Proses Interpreted Language dari Tokenisasi hingga Eksekusi

Sumber: Asisten

Pada Milestone I hingga III, seluruh proses berfokus pada fase Front-end, yaitu memotong teks sumber menjadi token, memvalidasi tata bahasanya, dan memastikan makna semantiknya benar. Hasil akhir dari fase tersebut adalah Decorated AST, sebuah struktur data murni yang sudah terbebas dari teks mentah dan telah dianotasi dengan informasi semantik seperti tipe data dan referensi ke tabel simbol.

Pada Milestone IV ini, proses memasuki fase Back-end sepenuhnya. Berbeda dengan Front-end, fase Back-end sama sekali tidak peduli dengan sintaksis bahasa Arion, apakah bahasa tersebut menggunakan titik koma, kurung kurawal, atau spasi. Back-end berfokus murni pada dua hal: Sintesis dan Eksekusi. Sintesis adalah proses mengonversi Decorated AST menjadi instruksi operasional standar yang disebut Intermediate Code, sedangkan Eksekusi adalah proses mengelola memori dan menjalankan instruksi tersebut pada mesin virtual (Interpreter).

Pemisahan arsitektur yang tegas antara Front-end dan Back-end ini sangat penting. Dengan desain ini, jika suatu saat bahasa Arion ingin dikembangkan dengan sintaks yang berbeda, cukup merombak Front-end-nya saja. Selama Front-end baru tersebut menghasilkan Decorated AST yang sama, mesin Back-end akan tetap dapat mengeksekusinya tanpa perlu diubah sedikit pun.

#### 2.2. Intermediate Code Generation

Setelah fase Front-end selesai memvalidasi program, tugas pertama Back-end adalah menerjemahkan Decorated AST ke dalam bentuk instruksi yang lebih sederhana dan siap dieksekusi. Proses ini disebut Intermediate Code Generation. Hasil dari proses ini adalah Intermediate Code (IC), yaitu representasi program yang berada di antara bahasa sumber (source code)

dan bahasa mesin, sehingga lebih mudah dianalisis dan diproses lebih lanjut.

Penggunaan Intermediate Code memberikan keuntungan berupa fleksibilitas arsitektur. Back-end tidak lagi bergantung pada sintaks bahasa sumber, dan jika diperlukan, IC yang sama dapat dieksekusi oleh berbagai jenis mesin atau dioptimasi sebelum dijalankan.

### 2.2.1. Three Address Code

Dari berbagai format Intermediate Code yang ada, format standar yang paling luas digunakan di industri adalah Three-Address Code (TAC). Dinamakan demikian karena setiap baris instruksinya dirancang sangat sederhana, di mana maksimal hanya melibatkan tiga alamat memori, biasanya terdiri dari dua operand sumber dan satu lokasi tujuan. Struktur dasar TAC terlihat seperti berikut.

hasil := operand1 operator operand2

Dalam bahasa tingkat tinggi, seorang programmer bebas menulis ekspresi matematis yang panjang dan kompleks. Namun, CPU tidak dapat menghitung semuanya sekaligus; ia harus memecahnya menjadi operasi-operasi biner yang sederhana. TAC menyimulasikan batasan ini dengan memanfaatkan variabel sementara, yang biasanya direpresentasikan sebagai t1, t2, t3, dan seterusnya.

Sebagai contoh, ekspresi  $x := (a + b) * (c - d)$  akan diratakan menjadi rangkaian instruksi TAC berikut.

t1 := a + b  
t2 := c - d  
t3 := t1 \* t2  
x := t3

Proses perataan ini dilakukan dengan cara mengevaluasi dari cabang terdalam pohon AST bergerak ke akar, sehingga menghasilkan barisan instruksi primitif yang sangat mudah dieksekusi oleh mesin.

### 2.2.2. Translasi Control Flow

Mengubah ekspresi matematika menjadi TAC relatif mudah karena polanya bersifat pasti. Tantangan terbesar dalam Intermediate Code Generation justru terletak pada perataan struktur control flow seperti IF-ELSE dan WHILE, karena keduanya melibatkan blok kode bersarang yang harus diubah menjadi daftar instruksi linier.

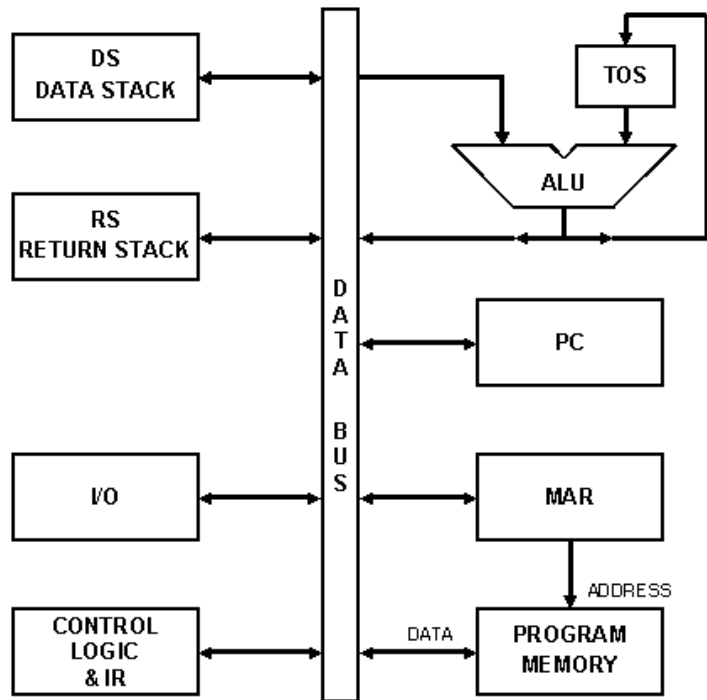
Solusi untuk masalah ini adalah dengan menggunakan kombinasi Label dan Jump (GOTO). Label adalah penanda statis yang menunjukkan posisi baris tertentu dalam TAC, misalnya L1: atau LABEL\_END:. Terdapat dua jenis Jump: Jump Bersyarat (IF\_FALSE ... GOTO ...) yang melompat ke suatu Label hanya jika kondisi yang dievaluasi bernilai false, dan Jump Mutlak (GOTO ...) yang melompat langsung ke suatu Label tanpa memperdulikan kondisi apa pun.

## **2.3. Interpreter**

Setelah Intermediate Code Generator selesai menghasilkan daftar instruksi TAC, pekerjaan translasi bahasa telah usai. Kini saatnya program benar-benar dieksekusi. Tugas ini diemban oleh Interpreter, yang bertindak sebagai sebuah Virtual Machine (VM). Interpreter akan membaca Intermediate Code baris demi baris, menyimpan nilai variabel ke dalam memori, mengevaluasi operasi, mengatur alur program, dan pada akhirnya menghasilkan output nyata.

Untuk melacak baris instruksi yang sedang dibaca, Interpreter menggunakan penunjuk khusus yang disebut Instruction Pointer (IP). Interpreter beroperasi dalam siklus eksekusi yang berulang: membaca instruksi yang ditunjuk IP (fetch), menguraikan perintah beserta operand-nya (decode), lalu melakukan aksi nyata seperti menghitung nilai atau menyimpan variabel ke memori (execute). Setelah setiap instruksi selesai dieksekusi, IP bergerak maju ke baris berikutnya, kecuali jika instruksi tersebut adalah GOTO, di mana IP akan langsung melompat ke Label yang dituju. Siklus ini terus berulang hingga IP mencapai akhir daftar instruksi TAC.

### **2.3.1. Memori Eksekusi Stack**



Gambar 2.3.1.1. Mekanisme Operasi Stack

Sumber: [https://users.ece.cmu.edu/~koopman/stack\\_computers/sec3\\_2.html](https://users.ece.cmu.edu/~koopman/stack_computers/sec3_2.html)

Untuk dapat melakukan perhitungan dan menyimpan variabel, VM membutuhkan memori. Dalam arsitektur interpreter, struktur memori yang paling vital adalah The Stack. Stack bekerja dengan prinsip LIFO (Last In, First Out) dan penggunaannya sangat erat kaitannya dengan konsep scope dan function call.

Setiap kali program memanggil sebuah fungsi atau memasuki blok baru, Interpreter membuat sebuah kotak memori baru di dalam Stack yang disebut Stack Frame. Di dalam frame inilah Interpreter menyimpan variabel lokal, argumen yang di-passing, serta return address. Saat fungsi dipanggil, Stack Frame barunya di-*push* ke puncak tumpukan, dan ketika fungsi selesai dieksekusi, frame tersebut di-*pop* sehingga semua variabel lokalnya otomatis musnah. Mekanisme ini menjelaskan penyebab variabel lokal di dalam suatu fungsi tidak dapat diakses dari fungsi lain, dan mengapa penggunaan memori program dapat tetap efisien.



## BAB 3

### PERANCANGAN

#### 3.1. Desain Intermediate Code Generation

|                  |                                                                                   |
|------------------|-----------------------------------------------------------------------------------|
| <b>Masukan</b>   | Decorated AST                                                                     |
| <b>Keluaran</b>  | Daftar instruksi bytecode dalam format PL/0 yang siap dieksekusi oleh Interpreter |
| <b>Algoritma</b> | <b>Depth First Search based approach</b>                                          |

Intermediate Code Generator (ICG) diimplementasikan dalam kelas `IntermediateCode`. Kelas ini menerima referensi `Symbol Table` dari tahap `Semantic Analysis` dan menggunakannya untuk melakukan pencarian informasi identifier, menghitung perbedaan lexical level, serta menentukan alamat memori variabel selama proses pembangkitan kode.

##### 3.1.1. Format Instruksi Bytecode

Instruksi yang dihasilkan ICG berbasis format PL/0 dengan struktur seragam berupa tiga field: opcode, level, dan target. Field level merepresentasikan perbedaan lexical level antara pemanggil dan tempat deklarasi variabel, sedangkan target menyimpan nilai operand seperti alamat memori, nilai literal, atau alamat lompatan. Himpunan instruksi yang didefinisikan adalah sebagai berikut.

*Tabel 3.1.1.1 Opcode yang Didefinisikan*

| Opcode | Fungsi                                                 |
|--------|--------------------------------------------------------|
| LIT    | Menempatkan nilai konstanta literal ke puncak Stack    |
| LOD    | Memuat nilai variabel dari Stack ke puncak Stack       |
| STO    | Menyimpan nilai dari puncak Stack ke variabel di Stack |
| CAL    | Memanggil prosedur atau fungsi pada alamat tertentu    |

|     |                                                                     |
|-----|---------------------------------------------------------------------|
| INT | Mengalokasikan ruang Stack Frame sebesar target slot                |
| JMP | Lompat tanpa syarat ke alamat instruksi target                      |
| JPC | Lompat ke target jika nilai puncak Stack adalah nol (false)         |
| OPR | Menjalankan operasi aritmatika, relasional, atau I/O sesuai OprCode |
| RET | Mengembalikan eksekusi ke pemanggil dan melepas Stack Frame         |

Operasi yang dijalankan oleh instruksi OPR ditentukan oleh nilai target yang bersesuaian dengan enum OprCode, meliputi operasi aritmatika (ADD, SUB, MUL, DIV, MOD), negasi (NEG), perbandingan (EQL, NEQ, LSS, LEQ, GTR, GEQ), serta operasi I/O (WRT, WRTLN).

### 3.1.2. Pembangkitan Kode Program

Proses pembangkitan kode dimulai dari method generate() yang menerima root AST bertipe ProgramNode. Jika program memiliki deklarasi prosedur atau fungsi, ICG terlebih dahulu memancarkan instruksi JMP untuk melompati blok-blok tersebut menuju kode utama. Setelah seluruh prosedur dan fungsi selesai dibangkitkan, instruksi JMP tersebut di-patch untuk menunjuk ke alamat kode utama. Program utama kemudian dibangkitkan dengan instruksi INT untuk mengalokasikan Stack Frame, diikuti isi compound statement, dan diakhiri dengan RET.

Setiap prosedur dan fungsi dibangkitkan oleh genProcDecl() dan genFuncDecl(). Alamat awal setiap prosedur/fungsi disimpan dalam procAddr agar dapat direferensikan ketika ada instruksi pemanggilan CAL. Selama pembangkitan blok prosedur atau fungsi, currentLevel dinaikkan satu untuk mencerminkan kedalaman lexical level yang baru, kemudian dikembalikan setelah blok selesai.

### 3.1.3. Pembangkitan Ekspresi

Proses pembangkitan kode dimulai dari method generate() yang menerima root AST bertipe ProgramNode. Jika program memiliki

deklarasi prosedur atau fungsi, ICG terlebih dahulu memancarkan instruksi JMP untuk melompati blok-blok tersebut menuju kode utama. Setelah seluruh prosedur dan fungsi selesai dibangkitkan, instruksi JMP tersebut di-patch untuk menunjuk ke alamat kode utama. Program utama kemudian dibangkitkan dengan instruksi INT untuk mengalokasikan Stack Frame, diikuti isi compound statement, dan diakhiri dengan RET.

Setiap prosedur dan fungsi dibangkitkan oleh `genProcDecl()` dan `genFuncDecl()`. Alamat awal setiap prosedur/fungsi disimpan dalam `procAddr` agar dapat direferensikan ketika ada instruksi pemanggilan CAL. Selama pembangkitan blok prosedur atau fungsi, `currentLevel` dinaikkan satu untuk mencerminkan kedalaman lexical level yang baru, kemudian dikembalikan setelah blok selesai.

#### **3.1.4. Pembangkitan Statement**

Pembangkitan pernyataan dilakukan oleh `genStmt()` yang mendispatch ke method yang sesuai berdasarkan tipe node.

Untuk `AssignNode`, nilai ekspresi sisi kanan dibangkitkan terlebih dahulu ke Stack, kemudian `genStore()` dipanggil untuk memilih instruksi penyimpanan yang tepat. Jika target adalah variabel biasa, dipancarkan STO; jika target adalah elemen array, indeks dibangkitkan dan STO diarahkan ke alamat dasar array.

Untuk `IfNode`, kondisi dibangkitkan dan diikuti JPC ke blok else (atau ke akhir jika tidak ada else). Jika terdapat blok else, setelah blok then selesai dipancarkan JMP untuk melompati blok else, kemudian alamat-alamat lompatan di-patch ke posisi yang tepat.

Untuk `WhileNode`, alamat awal kondisi dicatat sebagai `loopStart`. Setelah kondisi dibangkitkan, dipancarkan JPC sebagai syarat keluar. Setelah badan perulangan dibangkitkan, dipancarkan JMP kembali ke `loopStart`, dan JPC di-patch ke alamat setelah perulangan.

Untuk `ForNode`, nilai awal dievaluasi dan disimpan ke variabel kontrol. Pada setiap iterasi, variabel kontrol dimuat dan dibandingkan dengan nilai batas menggunakan LEQ (untuk to) atau GEQ (untuk downto). Setelah badan selesai dieksekusi, variabel kontrol diperbarui dengan penambahan atau pengurangan 1, kemudian lompat kembali ke awal kondisi.

Untuk RepeatNode, badan perulangan dibangkitkan terlebih dahulu, kemudian kondisi until dievaluasi. Instruksi JPC dipancarkan kembali ke awal perulangan selama kondisi bernilai false.

Untuk CaseNode, setiap cabang kasus dievaluasi dengan membandingkan ekspresi kunci terhadap setiap label menggunakan OPR EQL dan JPC. Jika tidak ada yang cocok, eksekusi melompat ke cabang berikutnya. Setelah setiap cabang dieksekusi, dipancarkan JMP ke akhir seluruh blok case.

Untuk CallStmtNode, prosedur standar writeln dan write ditangani secara khusus dengan memancarkan OPR WRTLN atau OPR WRT untuk setiap argumen. Pemanggilan prosedur yang didefinisikan pengguna dilakukan dengan membangkitkan seluruh argumen ke Stack terlebih dahulu, kemudian memancarkan instruksi CAL ke alamat prosedur yang tersimpan di procAddr.

### 3.1.5. Manajemen Lexical Level dan Alamat

ICG memelihara variabel currentLevel yang merepresentasikan kedalaman lexical level saat ini. Perbedaan level antara titik penggunaan dan titik deklarasi suatu variabel dihitung melalui fungsi levelDiff() yang mengembalikan selisih currentLevel dengan lev pada entri Symbol Table. Perbedaan ini digunakan sebagai argumen level pada instruksi LOD dan STO agar Interpreter dapat menelusuri static link pada Stack Frame yang tepat. Alamat variabel dihitung dengan menambahkan offset adr dari Symbol Table dengan 3 (untuk mengakomodasi tiga slot khusus dalam Stack Frame: static link, dynamic link, dan return address).

## 3.2. Desain Interpreter

|                  |                                                              |
|------------------|--------------------------------------------------------------|
| <b>Masukan</b>   | Daftar instruksi bytecode hasil pembangkitan ICG             |
| <b>Keluaran</b>  | Hasil eksekusi program berupa <i>output</i> ke <i>stream</i> |
| <b>Algoritma</b> | Stack Machine                                                |

Interpreter diimplementasikan dalam kelas Interpreter. Kelas ini menggunakan Stack berukuran tetap (STACK\_SIZE = 10000) yang direpresentasikan sebagai std::vector<int>. Tiga register utama digunakan selama eksekusi: pc (program counter) sebagai penunjuk instruksi aktif, sp

(stack pointer) sebagai penunjuk puncak Stack, dan bp (base pointer) sebagai penunjuk dasar Stack Frame aktif.

### 3.2.1. Siklus Eksekusi

Eksekusi dimulai dari method `execute()` yang menginisialisasi seluruh register ke kondisi awal ( $sp = -1$ ,  $bp = 0$ ,  $pc = 0$ ) dan menjalankan siklus fetch-decode-execute selama  $pc$  berada dalam rentang valid. Pada setiap iterasi, instruksi yang ditunjuk  $pc$  diambil,  $pc$  dinaikkan satu, kemudian instruksi didispatch ke handler yang sesuai melalui switch berdasarkan opcode-nya.

### 3.2.2. Siklus Instruksi

Berikut mekanisme eksekusi untuk setiap instruksi yang didukung Interpreter.

*Tabel 3.2.2.1 Instruksi yang Didukung Interpreter*

| Instruksi            | Deskripsi                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>execLIT</code> | menempatkan nilai target dari instruksi langsung ke puncak Stack melalui <code>push()</code>                                                                                                                                                                                                                                                                                                                                   |
| <code>execLOD</code> | memuat nilai variabel dari Stack. Alamat dasar frame yang tepat ditelusuri melalui <code>base(level)</code> menggunakan static link, kemudian nilai pada posisi <code>base + target</code> dimuat ke puncak Stack.                                                                                                                                                                                                             |
| <code>execSTO</code> | mengambil nilai dari puncak Stack melalui <code>pop()</code> dan menyimpannya ke posisi <code>base(level) + target</code> pada Stack.                                                                                                                                                                                                                                                                                          |
| <code>execCAL</code> | menangani pemanggilan prosedur atau fungsi. Sebelum melompat, tiga nilai ditulis ke Stack Frame baru: static link (hasil <code>base(level)</code> ), dynamic link (nilai <code>bp</code> saat ini), dan return address (nilai <code>pc</code> saat ini). Kemudian <code>bp</code> diperbarui ke posisi Stack Frame baru dan <code>pc</code> melompat ke alamat prosedur yang dituju. <code>callDepth</code> juga dinaikkan dan |

|         |                                                                                                                                                                                                                                                                                                                                        |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | dibatasi oleh MAX_CALL_DEPTH = 500 untuk mendeteksi infinite recursion.                                                                                                                                                                                                                                                                |
| execINT | mengalokasikan ruang pada Stack Frame dengan menaikkan sp sebesar target slot.                                                                                                                                                                                                                                                         |
| execJMP | memperbarui pc langsung ke alamat target tanpa syarat.                                                                                                                                                                                                                                                                                 |
| execJPC | mengambil nilai dari puncak Stack; jika nilai tersebut nol (false), pc diperbarui ke target. Jika tidak, eksekusi dilanjutkan secara sekuensial.                                                                                                                                                                                       |
| execOPR | mengeksekusi operasi yang ditentukan oleh target. Operasi aritmatika dan relasional mengambil dua nilai dari Stack, menghitung hasilnya, dan mendorongnya kembali. Operasi NEG membalik tanda nilai puncak Stack. Operasi WRT dan WRTLN mencetak nilai ke output stream, dengan WRTLN menambahkan karakter newline setelah pencetakan. |
| execRET | mengembalikan eksekusi ke pemanggil dengan memulihkan sp ke posisi sebelum Stack Frame (bp - 1), memulihkan pc dari return address, memulihkan bp dari dynamic link, dan menurunkan callDepth.                                                                                                                                         |

### 3.2.3. Penanganan Kesalahan Runtime

Interpreter mendeteksi dan menangani berbagai kondisi kesalahan selama eksekusi melalui kelas RuntimeError. Kondisi yang dideteksi meliputi Stack Overflow apabila sp melebihi STACK\_SIZE, Stack Underflow apabila terjadi pop() pada Stack kosong, Division by Zero pada operasi pembagian, Invalid Jump Target apabila pc diarahkan ke alamat di luar rentang valid, serta Out-of-bounds Stack Access apabila indeks Stack yang diakses berada di luar batas. Selain itu, kedalaman

pemanggilan fungsi dibatasi oleh MAX\_CALL\_DEPTH untuk mencegah infinite recursion yang menyebabkan Stack Overflow.

### 3.3. Teknologi

Berikut adalah teknologi yang digunakan dalam program:

*Tabel 3.2.1 Teknologi yang Digunakan*

| <b>Teknologi yang digunakan</b>        | <b>Penjelasan</b>                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bahasa Pemrograman C++ dan STL Library | Digunakan untuk mengimplementasikan lexer, DFA, parser, ASTer, semantic analyzer, intermediate code generator, interpreter, dan seluruh logika program. Program ini menggunakan library standar C++ seperti fstream untuk pengolahan file; map, set, dan vector untuk pengelolaan data, memory untuk manajemen node parse tree dan AST secara otomatis, serta unordered_map untuk type cache pada proses anotasi Decorated AST. |
| Build Tool Makefile                    | Digunakan untuk mengatur proses kompilasi program secara otomatis.                                                                                                                                                                                                                                                                                                                                                              |

### 3.4. Struktur Program

Berikut adalah struktur dari program:

MMD-TUBES-IF2224-2026/

```

|
|— doc/
|— src/
|   |— main.cpp
|   |— Symbol.cpp
|   |— Symbol.hpp
|   |— Token.cpp
|   |— Token.hpp
|   |
|   |— Lexer/
|   |   |— dfa_rules.txt

```

```

| | | — DFA.cpp
| | | — DFA.hpp
| | | — lexer.cpp
| | | — lexer.hpp
|
| — Parser/
| | — Parser.cpp
| | — Parser.hpp
| | — ParseTree.cpp
| | — ParseTree.hpp
|
| — Semantic/
| | — AST.cpp
| | — AST.hpp
| | — ASTer.cpp
| | — ASTer.hpp
| | — SemanticAnalyzer.cpp
| | — SemanticAnalyzer.hpp
| | — SymbolTable.cpp
| | — SymbolTable.hpp
|
| — Interpreter/
| | — Instructions.hpp
| | — ICG.cpp
| | — ICG.hpp
| | — Interpreter.cpp
| | — Interpreter.hpp
| | — Stack.hpp
|
| — test/
| | — milestone-1/
| | — milestone-2/
| | — milestone-3/
| | — milestone-4/
|
| — README.md
| — Makefile

```



### 3.5. Fungsi Utama

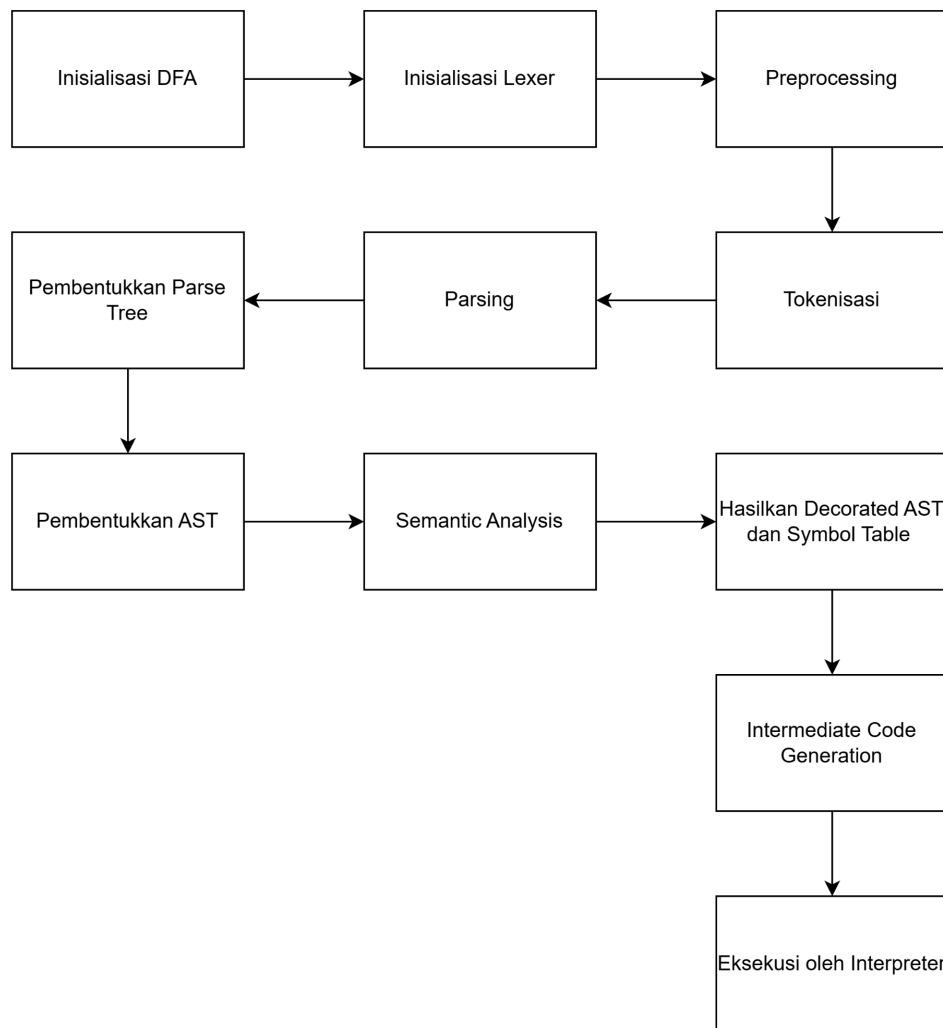
Tabel 3.5.1 Desain Fungsi Utama

| Nama Kelas       | Data/Method             | Penjelasan                                                                                                                                        |
|------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| IntermediateCode | generate(root)          | Method publik utama yang memulai proses pembangkitan kode dari root AST. Mengembalikan vector<Bytecode> berisi seluruh instruksi yang dihasilkan. |
|                  | genProgram(node)        | Membangkitkan kode untuk seluruh program, mencakup deklarasi prosedur/fungsi dan blok utama.                                                      |
|                  | genProcDecl(node)       | Membangkitkan kode untuk deklarasi prosedur, mencatat alamat awal ke procAddr, dan mengatur perubahan currentLevel.                               |
|                  | genFuncDecl(node)       | Membangkitkan kode untuk deklarasi fungsi, sama dengan genProcDecl dengan tambahan nilai kembalian.                                               |
|                  | genStmt(node)           | Dispatcher utama untuk pernyataan, mendelegasikan ke genAssign, genIf, genWhile, genFor, genRepeat, genCase, atau genCallStmt.                    |
|                  | genExpr(node)           | Dispatcher utama untuk ekspresi, mendelegasikan ke genLiteral, genVarRef, genBinaryOp, genUnaryOp, genCall, atau genArrayAccess.                  |
|                  | genStore(target)        | Menentukan instruksi STO yang tepat berdasarkan jenis target (variable atau array element).                                                       |
|                  | emit(op, level, target) | Memancarkan satu instruksi bytecode ke daftar code dan mengembalikan indeksinya.                                                                  |

|             |                                           |                                                                                                                                                                                    |
|-------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             | <code>patch(index, newTarget)</code>      | Memperbarui field target pada instruksi di posisi <code>index</code> , digunakan untuk menyelesaikan alamat lompatan yang belum diketahui saat instruksi pertama kali dipancarkan. |
|             | <code>levelDiff(tabIdx)</code>            | Menghitung perbedaan lexical level antara titik penggunaan dan titik deklarasi suatu variabel.                                                                                     |
|             | <code>frameSizeFromBlock(blockIdx)</code> | Menghitung ukuran Stack Frame berdasarkan informasi pada <code>btab</code> : $3 + vsze + psze$ .                                                                                   |
| Interpreter | <code>execute(code)</code>                | Method publik yang menginisialisasi register dan menjalankan siklus eksekusi hingga seluruh instruksi selesai diproses.                                                            |
|             | <code>execLIT(instr)</code>               | Mendorong nilai literal ke puncak Stack.                                                                                                                                           |
|             | <code>execLOD(instr)</code>               | Memuat nilai variabel dari Stack Frame yang tepat ke puncak Stack.                                                                                                                 |
|             | <code>execSTO(instr)</code>               | Menyimpan nilai dari puncak Stack ke variabel pada Stack Frame yang tepat.                                                                                                         |
|             | <code>execCAL(instr, code)</code>         | Menyiapkan Stack Frame baru dan melompat ke alamat prosedur/fungsi.                                                                                                                |
|             | <code>execINT(instr)</code>               | Mengalokasikan ruang pada Stack dengan menaikkan <code>sp</code> .                                                                                                                 |
|             | <code>execJMP(instr)</code>               | Memperbarui <code>pc</code> ke alamat lompatan tanpa syarat.                                                                                                                       |
|             | <code>execJPC(instr)</code>               | Memperbarui <code>pc</code> ke alamat lompatan jika nilai puncak Stack adalah nol.                                                                                                 |

|  |                                  |                                                                                                           |
|--|----------------------------------|-----------------------------------------------------------------------------------------------------------|
|  | <code>execOPR(instr, out)</code> | Mengeksekusi operasi aritmatika, relasional, atau I/O sesuai OprCode.                                     |
|  | <code>execRET()</code>           | Memulihkan register sp, pc, dan bp ke kondisi sebelum pemanggilan dan menurunkan callDepth.               |
|  | <code>base(levelDiff)</code>     | Menelusuri static link pada Stack sebanyak levelDiff kali untuk menemukan base pointer frame yang dituju. |

### 3.6. Alur Kerja Program



Gambar 3.6.1 Alur Kerja Program

Secara umum, program bekerja melalui empat tahap utama, yaitu lexical analysis, syntax analysis, semantic analysis, dan intermediate code generation beserta interpretation. Pada tahap lexical analysis, source code diubah menjadi token menggunakan DFA. Selanjutnya, parser melakukan recursive descent parsing untuk membentuk parse tree berdasarkan grammar bahasa. Setelah parse tree berhasil dibentuk, semantic analyzer akan mengonversi parse tree menjadi Abstract Syntax Tree (AST), kemudian melakukan semantic checking untuk menghasilkan Decorated AST beserta Symbol Table. Komponen utama yang digunakan meliputi DFA, Lexer, Parser, AST Builder, Semantic Analyzer, serta Symbol Table. Pada tahap terakhir, Intermediate Code Generator meratakan Decorated AST menjadi instruksi TAC yang kemudian dieksekusi oleh Interpreter. Komponen utama yang digunakan meliputi DFA, Lexer, Parser,

AST Builder, Semantic Analyzer, Symbol Table, Intermediate Code Generator, serta Interpreter.

Berikut penjelasan alur program secara keseluruhan:

### 1. Inisialisasi DFA

Pada tahap ini, program membaca file aturan DFA (dfa\_rules.txt). Semua aturan transisi akan dimuat ke dalam struktur transition.

### 2. Inisialisasi Lexer

Pada tahap ini, lexer membuka file source code, kemudian membaca karakter per karakter.

### 3. Preprocessing

Lexer akan melewati spasi, tab, newline, sekaligus melakukan update nomor baris. Dilakukan supaya DFA hanya memproses karakter yang relevan.

### 4. Proses Tokenisasi

Merupakan loop utama dari program. Menggunakan fungsi tokenize() yang didefinisikan di lexer.cpp. Dengan alur sebagai berikut:

a) Mulai dari

currentState = start\_state

lexeme = ""

b) Untuk setiap karakter,

Ambil next state:

nextState = getNextState(...)

c) Terdapat tiga kemungkinan kasus:

#### **CASE 1: Transisi Valid**

Lanjut membangun token. Ini mencakup transisi spesifik dan wildcard.

lexeme += currentChar;

currentState = nextState;

#### **CASE 2: Masuk Dead State**

Terjadi apabila tidak ada transisi spesifik dan tidak ada wildcard.

if(DFA.isFinalState(currentState))

o Jika sebelumnya final:

simpan token

reset state

o Jika tidak:

error / stop

d) EOF Handling

Jika masih ada lexeme dan state final, maka tetap jadi token.

e) Output

Menghasilkan vector<Token>.

## **5. Parsing**

Setelah proses tokenisasi selesai, vector token dikirim ke parser. Parser menggunakan metode recursive descent parsing untuk memeriksa kesesuaian token terhadap grammar bahasa. Apabila ditemukan token yang tidak sesuai dengan grammar, parser akan mengembalikan nullptr dan menampilkan informasi syntax error berupa token yang diharapkan, token yang diterima, serta nomor baris error.

## **6. Pembentukan Parse Tree**

Selama proses parsing, parser membangun struktur parse tree menggunakan TreeNode. Setiap non-terminal grammar direpresentasikan sebagai node yang memiliki child sesuai hasil parsing produksi grammar. Parse tree hanya akan dihasilkan apabila seluruh proses parsing berhasil dan source code valid secara sintaks.

## **7. Pembentukan AST**

Program melakukan konversi parse tree menjadi Abstract Syntax Tree (AST) menggunakan pendekatan Syntax-Directed Translation (SDT). Dengan cara, pada setiap aturan produksi grammar, ditambahkan semantic action yang bertugas membangun node AST yang bersesuaian. Selama proses ini, elemen-elemen sintaksis yang tidak lagi diperlukan untuk analisis semantik dihilangkan. Sehingga, hasilnya adalah struktur pohon yang lebih ringkas dan berfokus pada struktur semantik program (AST).

## **8. Semantic Analysis**

Akan dilakukan traversal pada AST yang telah terbentuk secara DFS menggunakan fungsi visitor yang akan melakukan pengecekan semantic program. Di antaranya: lookup identifier pada Symbol Table, pengecekan deklarasi variabel, pengecekan scope, inferensi tipe data, validasi kompatibilitas tipe, serta anotasi semantic pada node AST.

## **9. Hasilkan Decorated AST dan Symbol Table**

Hasil akhir program berupa Decorated Abstract Syntax Tree (DAST), Symbol Table (tab, atab, btab), serta informasi semantic error apabila ditemukan pelanggaran aturan semantic program.

## **10. Intermediate Code Generation**

Decorated AST yang telah dihasilkan kemudian diratakan menjadi daftar instruksi Three-Address Code (TAC) oleh Intermediate Code Generator. Proses ini dilakukan dengan melakukan traversal pada Decorated AST secara post-order, di mana setiap node ditranslasikan menjadi satu atau beberapa instruksi TAC. Ekspresi kompleks dipecah menggunakan variabel sementara, sedangkan struktur control flow seperti IF-ELSE dan WHILE ditranslasikan menggunakan kombinasi Label dan instruksi GOTO.

## **11. Eksekusi oleh Interpreter**

Daftar instruksi TAC yang telah dihasilkan kemudian dieksekusi oleh Interpreter yang bertindak sebagai Virtual Machine. Interpreter membaca instruksi satu per satu menggunakan Instruction Pointer (IP), mengevaluasi setiap operasi, menyimpan nilai variabel ke dalam memori melalui mekanisme Stack, serta mengatur alur eksekusi program berdasarkan instruksi GOTO dan Label. Proses ini terus berlangsung hingga seluruh instruksi TAC selesai dieksekusi dan program menghasilkan output akhir.

### 3.7. Pendekatan Implementasi

Implementasi milestone ini terbagi menjadi dua komponen utama: Intermediate Code Generation (ICG) yang mengonversi Decorated AST menjadi instruksi bytecode, dan Interpreter yang mengeksekusi bytecode tersebut pada sebuah mesin virtual berbasis stack. Pada komponen ICG, desain mengikuti pola generator rekursif top-down yang mencerminkan struktur AST secara langsung. Setiap jenis node AST dipetakan ke satu method `genXxx()` yang bertanggung jawab penuh atas penerbitan instruksi untuk node tersebut, sehingga perubahan pada satu jenis statement atau ekspresi tidak berdampak pada generator lainnya. Titik masuk tunggal `generate()` memulai penelusuran dari `ProgramNode`, dan setiap `genStmt()` serta `genExpr()` mendelegasikan ke sub-method yang tepat berdasarkan `ASTType` node, konsisten dengan pola dispatch yang sudah digunakan pada fase analisis semantik.

Tata letak stack frame dirancang dengan tiga slot overhead di awal frame (static link, dynamic link, return address di posisi `bp+0`, `bp+1`, `bp+2`), diikuti slot return value fungsi di `bp+3` (konstanta `FUNC_RETVAL_SLOT`), kemudian argumen dan variabel lokal mulai dari offset `bp+4`. Pilihan ini memungkinkan penentuan alamat variabel dilakukan secara statis pada waktu kompilasi menggunakan rumus `adr + 3` dari symbol table, tanpa perlu informasi runtime. Pencarian variabel lintas scope diimplementasikan dengan `lookupVar()` yang menelusuri `blockStack` dari level terdalam ke terluar, lalu menghitung perbedaan level leksikal (`levelDiff`) untuk menghasilkan operan level yang benar pada instruksi LOD/STO.

Untuk struktur kontrol, setiap pola aliran dibangkitkan dengan teknik backpatching: alamat lompatan diterbitkan terlebih dahulu dengan nilai placeholder menggunakan `emit()`, kemudian ditambal ke alamat sebenarnya melalui `patch()` setelah blok target selesai dikodekan. Pendekatan ini menghindari dua kali penelusuran AST dan menjaga pembangkitan kode tetap linear dalam satu lintasan. Operator `and` dan `or` diimplementasikan dengan short-circuit evaluation menggunakan serangkaian JPC dan JMP, sehingga sisi kanan ekspresi hanya dievaluasi bila diperlukan, sesuai semantik Pascal.

Akses array multi-dimensi dan record ditangani melalui sepasang instruksi indirect LODI/STOI. Untuk array, collectArrayAccessInfo() meratakan rantai ArrayAccessNode bersarang menjadi daftar indeks dan nilai pengali yang dihitung dari metadata atab[] symbol table, kemudian membangkitkan sekuens aritmatika di stack untuk menghasilkan offset akhir secara runtime. Pendekatan indirect ini dipilih karena offset array tidak dapat diketahui secara statis; sementara akses field record tetap menggunakan LOD/STO langsung karena offset field sudah diketahui saat kompilasi. Nilai bertipe real dan string tidak disimpan langsung di stack melainkan diinternkan ke tabel terpisah (realValues, stringTable) dan direpresentasikan sebagai tagged integer menggunakan bit-mask (REAL\_TAG, STRING\_TAG), sehingga slot stack tetap berukuran seragam (satu int) tanpa memerlukan union atau void pointer.

Pada komponen Interpreter, mesin virtual diimplementasikan sebagai fetch-decode-execute loop sederhana yang mengiterasi array instruksi berdasarkan program counter (pc). Semua state runtime (stack, sp, bp, pc, callDepth) disimpan sebagai field kelas, sehingga tidak ada state global dan pengujian ulang eksekusi dapat dilakukan tanpa membuat instance baru. Instruksi CAL diimplementasikan dengan mengeluarkan argumen dari stack terlebih dahulu, lalu mendorong tiga slot overhead, mengatur bp baru, dan memindahkan argumen kembali ke frame yang baru terbentuk. Mekanisme ini mempertahankan konvensi bahwa argumen selalu terletak di bp+4 dan seterusnya, sehingga ICG dan Interpreter berbagi asumsi tata letak frame yang sama tanpa perlu koordinasi runtime tambahan. Nilai kembali fungsi disimpan sementara di stack[bp+3] sebelum frame dibongkar oleh execRET(), lalu didorong kembali ke stack oleh execRETVAL() segera setelah CAL selesai, sehingga ekspresi pemanggil dapat langsung menggunakan nilai tersebut sebagai operan berikutnya.

Seluruh kondisi error runtime seperti stack overflow, stack underflow, pembagian nol, lompatan ke alamat tidak valid, akses stack di luar batas, ditangani oleh method pemeriksaan terpusat yang melempar RuntimeError. Pemisahan ini menjaga setiap executor (execXxx) tetap bersih dari logika pengecekan dan memudahkan penambahan pemeriksaan baru di masa mendatang tanpa memodifikasi logika eksekusi itu sendiri.



## BAB 4

### HASIL IMPLEMENTASI

#### Class Bytecode:

```
class Bytecode {
private:
    int index;
    Opcode opcode;
    int level;
    int target;
public:
    Bytecode(int index, Opcode op, int lev, int oper)
        : index(index), opcode(op), level(lev), target(oper) {}
    int getIndex() { return index; }
    Opcode getOp() { return opcode; }
    int getLevel() { return level; }
    int getTarget() { return target; }
    void setTarget(int newTarget) { this->target = newTarget; }
    std::string opcodeToStr() const {
        switch (opcode) {
            case Opcode::LIT:    return "LIT";
            case Opcode::LITS:   return "LITS";
            case Opcode::LITR:   return "LITR";
            case Opcode::LOD:    return "LOD";
            case Opcode::STO:    return "STO";
            case Opcode::CAL:    return "CAL";
            case Opcode::INT:    return "INT";
            case Opcode::JMP:    return "JMP";
            case Opcode::JPC:    return "JPC";
            case Opcode::OPR:    return "OPR";
            case Opcode::RET:    return "RET";
            case Opcode::RETVAL: return "RETVAL";
            case Opcode::LODI:   return "LODI";
            case Opcode::STOI:   return "STOI";
            default: return "???";
        }
    }
    std::string toString(int lineNum) const {
```

```

        return std::to_string(lineNum) + " "
               + opcodeToStr() + " "
               + std::to_string(level) + " "
               + std::to_string(target);
    }
};

```

#### Penjelasan:

Kelas Bytecode dan enum Opcode serta OprCode didefinisikan dalam Instructions.hpp sebagai representasi satu instruksi bytecode. Setiap objek Bytecode menyimpan empat field: index (nomor urut instruksi), opcode (jenis operasi), level (perbedaan lexical level), dan target (nilai operand). Kelas ini juga menyediakan method toString() untuk mencetak instruksi dalam format yang dapat dibaca, serta setTarget() untuk keperluan patching alamat lompatan.

#### Class IntermediateCode:

```

class IntermediateCode {
private:
    SymbolTable& symTab;
    std::vector<Bytecode> code;
    int currentLevel;

    std::vector<int> blockStack;

    std::string currentFuncName;
    int currentFuncTabIdx;

    int emit(Opcode op, int level, int target) {
        int index = code.size();
        Bytecode newLine(index, op, level, target);
        code.push_back(newLine);
        return index;
    }

    void patch(int index, int newTarget) {
        if (index < 0 || index >= static_cast<int>(code.size()))
            throw std::runtime_error("ICG: patch ke alamat tidak valid: "
+ std::to_string(index));
        code.at(index).setTarget(newTarget);
    }
}

```

```

int size() { return code.size(); }
Bytecode& get(int index) { return code[index]; }
int nextAddr() const { return static_cast<int>(code.size()); }

void genProgram(std::shared_ptr<ProgramNode> node);
void genProcDecl(std::shared_ptr<ProcDeclNode> node);
void genFuncDecl(std::shared_ptr<FuncDeclNode> node);

void genStmt(std::shared_ptr<StmtNode> node);
void genCompound(std::shared_ptr<CompoundNode> node);
void genAssign(std::shared_ptr<AssignNode> node);
void genIf(std::shared_ptr<IfNode> node);
void genWhile(std::shared_ptr<WhileNode> node);
void genFor(std::shared_ptr<ForNode> node);
void genRepeat(std::shared_ptr<RepeatNode> node);
void genCase(std::shared_ptr<CaseNode> node);
void genCallStmt(std::shared_ptr<CallStmtNode> node);

void genExpr(std::shared_ptr<ExprNode> node);
void genLiteral(std::shared_ptr<LiteralNode> node);
void genVarRef(std::shared_ptr<VarRefNode> node);
void genBinaryOp(std::shared_ptr<BinaryOpNode> node);
void genUnaryOp(std::shared_ptr<UnaryOpNode> node);
void genCall(std::shared_ptr<CallNode> node);
void genArrayAccess(std::shared_ptr<ArrayAccessNode> node);
void genFieldAccess(std::shared_ptr<FieldAccessNode> node);
void genStore(std::shared_ptr<ExprNode> target);

int lookupVar(const std::string& name) const;
int levelDiff(int tabIdx) const;

int frameSize(std::shared_ptr<ProcDeclNode> node) const;
int frameSize(std::shared_ptr<FuncDeclNode> node) const;
int frameSizeFromBlock(int blockIdx) const;
int findProcBtab(const std::string& name, int objType) const;
    int getRecordFieldAddr(int recTabIdx, const std::string& fieldName)
const;
int getArrayElementSlotCount(int arrayRef) const;
bool collectArrayAccessInfo(std::shared_ptr<ArrayAccessNode> node,
                           int& baseIdx,

```

```

                                std::vector<std::shared_ptr<ExprNode>>&
indices,

                                std::vector<int>& lowers,
                                std::vector<int>& multipliers,
                                int& fieldOffset) const;

    std::unordered_map<std::string, int> procAddr;

    int varAddr(int tabIdx) const { return symTab.getTab(tabIdx).adr + 3;
}

    Opcode opToOpr(const std::string& op) const;

    std::vector<std::string> stringTable;
    int internString(const std::string& s) {
        for (int i = 0; i < (int)stringTable.size(); i++)
            if (stringTable[i] == s) return i;
        stringTable.push_back(s);
        return (int)stringTable.size() - 1;
    }

    std::vector<double> realValues;
    int internReal(double r) {
        for (int i = 0; i < (int)realValues.size(); i++)
            if (realValues[i] == r) return i;
        realValues.push_back(r);
        return (int)realValues.size() - 1;
    }

public:
    IntermediateCode(SymbolTable& symTab)
        : symTab(symTab), currentLevel(0), currentFuncTabIdx(-1) {}

    std::vector<Bytecode> generate(std::shared_ptr<ASTNode> root);

    void writeToFile(const std::vector<Bytecode>& instrs, const
std::string& filename) const;
    void writeToStream(const std::vector<Bytecode>& instrs, std::ostream&
out) const;
    const std::vector<std::string>& getStringTable() const { return
stringTable; }

```

```
const std::vector<double>& getRealValues() const { return realValues;
}
};
```

#### Penjelasan:

Kelas IntermediateCode merupakan implementasi dari ICG. Atribut utamanya adalah code (daftar instruksi yang sedang dibangun), symTab (referensi ke Symbol Table), currentLevel (kedalaman lexical level aktif), dan procAddr (peta nama prosedur/fungsi ke alamat instruksi awalnya).

Kelas IntermediateCode bertindak sebagai generator yang mengubah Decorated AST menjadi daftar instruksi bytecode secara rekursif. Method emit() menambahkan satu instruksi baru ke code dan mengembalikan indeksinya untuk keperluan patching di kemudian hari. Method patch() memperbarui field target pada instruksi tertentu, yang digunakan untuk menyelesaikan alamat lompatan yang belum diketahui ketika instruksi pertama kali dipancarkan, misalnya pada pembangkitan struktur IF-ELSE dan WHILE. Seluruh method genXxx() bersifat privat dan hanya dapat diakses melalui generate() sebagai titik masuk utama.

#### Class Interpreter:

```
class Interpreter {
public:
    static const int STACK_SIZE = 10000;
    static const int MAX_CALL_DEPTH = 500;
    Interpreter() : stack(STACK_SIZE, 0), sp(-1), bp(0), pc(0),
callDepth(0), returnValue(0), lastCallNArgs(0) {}
    void setStringTable(const std::vector<std::string>& st) { stringTable
= st; }
    void setRealValues(const std::vector<double>& rv) { realValues = rv; }
    void execute(std::vector<Bytecode>& code);
    void execute(std::vector<Bytecode>& code, std::ostream& out);

private:
    std::vector<int> stack;
    std::vector<std::string> stringTable;
    std::vector<double> realValues;
    int sp;
    int bp;
    int pc;
    int callDepth;
    int returnValue; // saved return value from the last function call
    int lastCallNArgs; // number of args of the last CAL (saved at CAL
```

time, used by RETVAL)

```
void push(int val);
int pop();
int top() const;

int base(int levelDiff) const;

void printVal(int val, std::ostream& out) const;

void execLIT(Bytecode instr);
void execLITS(Bytecode instr);
void execLITR(Bytecode instr);
void execLOD(Bytecode instr);
void execSTO(Bytecode instr);
void execCAL(Bytecode instr, const std::vector<Bytecode>& code);
void execINT(Bytecode instr);
void execJMP(Bytecode instr);
void execJPC(Bytecode instr);
void execOPR(Bytecode instr, std::ostream& out);
void execRET();
void execRETVAl();
void execLODI(Bytecode instr);
void execSTOI(Bytecode instr);

// Helper for real arithmetic
double decodeValue(int val) const;
int encodeRealResult(double val);

void checkStackOverflow() const;
void checkStackUnderflow() const;
void checkDivision(int divisor) const;
void checkJumpTarget(int target, int codeSize) const;
void checkStackIndex(int idx) const;
};
```

**Penjelasan:**

Kelas Interpreter mengimplementasikan mesin eksekusi berbasis Stack. Atribut utamanya adalah stack (array memori berukuran tetap), sp, bp, pc sebagai register mesin, dan callDepth sebagai penghitung kedalaman pemanggilan fungsi.

Kelas Interpreter mengeksekusi instruksi bytecode satu per satu menggunakan siklus fetch-decode-execute. Register pc berperan sebagai Instruction Pointer yang menunjuk instruksi aktif, sp menunjuk puncak Stack, dan bp menunjuk dasar Stack Frame aktif. Method base() menelusuri static link pada Stack sebanyak levelDiff kali untuk menemukan frame yang tepat saat mengakses variabel dari scope yang berbeda. Setiap kondisi kesalahan runtime ditangkap dan dilempar sebagai RuntimeError agar program tidak crash tanpa informasi yang jelas.

### Class RuntimeError:

```
class RuntimeError : public std::runtime_error {
public:
    explicit RuntimeError(const std::string& msg)
        : std::runtime_error("Runtime Error: " + msg) {}
};

static constexpr int STRING_TAG = 0x40000000;
static constexpr int REAL_TAG  = 0x20000000;

inline bool isStringVal(int v) { return v >= 0 && (v & STRING_TAG) != 0; }
inline int  makeStringVal(int i) { return STRING_TAG | i; }
inline int  getStringIdx(int v) { return v & ~STRING_TAG; }

inline bool isRealVal(int v) { return v >= 0 && (v & REAL_TAG) != 0; }
inline int  makeRealVal(int i) { return REAL_TAG | i; }
inline int  getRealIdx(int v) { return v & ~REAL_TAG; }
```

### Penjelasan:

Penggunaan kelas terpisah untuk kesalahan runtime memungkinkan pemanggil untuk menangkap (catch) kesalahan eksekusi secara spesifik dan terpisah dari kesalahan umum lainnya. Kondisi yang terdeteksi meliputi Stack Overflow, Stack Underflow, Division by Zero, Invalid Jump Target, dan Out-of-bounds Stack Access.

## BAB 5

### EKSPERIMEN

#### ❖ Kasus uji 1

*Tabel 5.1 Kasus Uji 1*

| Input                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> program Hello; var   a, b: integer; begin   a := 5;   b := a + 10;   writeln('Result = ', b); end. </pre>                                                                                          |
| Output                                                                                                                                                                                                   |
| <pre> === INTERMEDIATE CODE === 0 INT 0 5 1 LIT 0 5 2 STO 0 3 3 LOD 0 3 4 LIT 0 10 5 OPR 0 2 6 STO 0 4 7 LITS 0 0 8 OPR 0 13 9 LOD 0 4 10 OPR 0 14 11 RET 0 0  === OUTPUT PROGRAM === Result = 15 </pre> |

#### ❖ Kasus uji 2

*Tabel 5.2 Kasus Uji 2*

| Input                                                                         |
|-------------------------------------------------------------------------------|
| <pre> program ValidTest; var   x, y: integer;   z: real;   b: boolean; </pre> |



```

arr: array [1..10] of integer;
rec: record
    val: integer;
    flag: boolean
end;

function add(val1: integer; val2: integer): integer;
var temp: integer;
begin
    temp := val1 + val2;
    add := temp;
end;

begin
    x := 5;
    y := 10;
    z := 1.3;
    b := (x < y) and true;

    arr[1] := add(x, y);
    rec.val := 100;
    rec.flag := false;

    writeln('FUNC CALL TEST');
    writeln('Result 1 = ', x + y);
    writeln('Result 2 = ', add(x, y));
    writeln('Result 3 = ', x - y);
    writeln('Result 4 = ', add(x, -y));
    writeln('Result 5 = ', x * y);

    writeln('REAL OUTPUT TEST');
    writeln('Result 6 = ', z);
    writeln('Result 7 = ', z * 10);
    writeln('Result 8 = ', z + 10);

    if b then
        writeln('Valid')
    else
        x := 0;

    writeln('LOOP TEST');
    while x > 0 do
    begin
        x := x - 1;
        writeln('x = ', x);
    end;

end.

```

## Output

=== INTERMEDIATE CODE ===

```
0 JMP 0 9
1 INT 0 7
2 LOD 0 4
3 LOD 0 5
4 OPR 0 2
5 STO 0 6
6 LOD 0 6
7 STO 0 3
8 RET 0 0
9 INT 0 19
10 LIT 0 5
11 STO 0 3
12 LIT 0 10
13 STO 0 4
14 LITR 0 0
15 STO 0 5
16 LOD 0 3
17 LOD 0 4
18 OPR 0 9
19 JPC 0 24
20 LIT 0 1
21 JPC 0 24
22 LIT 0 1
23 JMP 0 25
24 LIT 0 0
25 STO 0 6
26 LOD 0 3
27 LOD 0 4
28 CAL 2 1
29 RETVAL 0 2
30 LIT 0 7
31 LIT 0 1
32 LIT 0 1
33 OPR 0 3
34 OPR 0 2
35 STOI 0 0
36 LIT 0 100
37 STO 0 17
38 LIT 0 0
39 STO 0 18
40 LITS 0 0
41 OPR 0 14
42 LITS 0 1
43 OPR 0 13
44 LOD 0 3
```

45 LOD 0 4  
46 OPR 0 2  
47 OPR 0 14  
48 LITS 0 2  
49 OPR 0 13  
50 LOD 0 3  
51 LOD 0 4  
52 CAL 2 1  
53 RETVAL 0 2  
54 OPR 0 14  
55 LITS 0 3  
56 OPR 0 13  
57 LOD 0 3  
58 LOD 0 4  
59 OPR 0 3  
60 OPR 0 14  
61 LITS 0 4  
62 OPR 0 13  
63 LOD 0 3  
64 LOD 0 4  
65 OPR 0 1  
66 CAL 2 1  
67 RETVAL 0 2  
68 OPR 0 14  
69 LITS 0 5  
70 OPR 0 13  
71 LOD 0 3  
72 LOD 0 4  
73 OPR 0 4  
74 OPR 0 14  
75 LITS 0 6  
76 OPR 0 14  
77 LITS 0 7  
78 OPR 0 13  
79 LOD 0 5  
80 OPR 0 14  
81 LITS 0 8  
82 OPR 0 13  
83 LOD 0 5  
84 LIT 0 10  
85 OPR 0 4  
86 OPR 0 14  
87 LITS 0 9  
88 OPR 0 13  
89 LOD 0 5  
90 LIT 0 10  
91 OPR 0 2  
92 OPR 0 14

```
93 LOD 0 6
94 JPC 0 98
95 LITS 0 10
96 OPR 0 14
97 JMP 0 100
98 LIT 0 0
99 STO 0 3
100 LITS 0 11
101 OPR 0 14
102 LOD 0 3
103 LIT 0 0
104 OPR 0 11
105 JPC 0 115
106 LOD 0 3
107 LIT 0 1
108 OPR 0 3
109 STO 0 3
110 LITS 0 12
111 OPR 0 13
112 LOD 0 3
113 OPR 0 14
114 JMP 0 102
115 RET 0 0

=== OUTPUT PROGRAM ===
FUNC CALL TEST
Result 1 = 15
Result 2 = 15
Result 3 = -5
Result 4 = -5
Result 5 = 50
REAL OUTPUT TEST
Result 6 = 1.3
Result 7 = 13
Result 8 = 11.3
Valid
LOOP TEST
x = 4
x = 3
x = 2
x = 1
x = 0
```

❖ Kasus uji 3

Tabel 5.3 Kasus Uji 3

| Input                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> program TestSubprograms; var     result: integer;  function square(n: integer): integer; var     s: integer; begin     s := n * n;     square := s; end;  procedure printDouble(val: real); begin     writeln(val * 2); end;  begin     result := square(7);     writeln(result);     printDouble(6.7);     printDouble(result); end. </pre> |
| Output                                                                                                                                                                                                                                                                                                                                             |
| <pre> === INTERMEDIATE CODE === 0 JMP 0 15 1 INT 0 5 2 LOD 0 4 3 LIT 0 2 4 OPR 0 4 5 OPR 0 14 6 RET 0 0 7 INT 0 6 8 LOD 0 4 9 LOD 0 4 10 OPR 0 4 11 STO 0 5 12 LOD 0 5 13 STO 0 3 14 RET 0 0 15 INT 0 4 16 LIT 0 7 17 CAL 1 7 </pre>                                                                                                               |

```

18 RETVAL 0 1
19 STO 0 3
20 LOD 0 3
21 OPR 0 14
22 LITR 0 0
23 CAL 1 1
24 LOD 0 3
25 CAL 1 1
26 RET 0 0

=== OUTPUT PROGRAM ===
49
13.4
98

```

❖ Kasus uji 4

*Tabel 5.4 Kasus Uji 4*

| Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> program Hello; var   a : array [1..10] of array [1..10] of integer;   b : integer;  function factorial(n: integer): integer; begin   if (n == 0) then     factorial := 1   else     factorial := n * factorial(n - 1); { &lt;--- Panggilan rekursif } end;  begin   a[1][1] := 3;   a[4][4] := 11;   a[2][3] := -20;   a[2][3] := a[2][3] + a[1][1] + a[4][4];    writeln(a[2][3]);    { Mari kita uji dengan angka 5 (seharusnya mencetak 120) }   writeln(factorial(5)); end. </pre> |

## Output

=== INTERMEDIATE CODE ===

```
0 JMP 0 18
1 INT 0 5
2 LOD 0 4
3 LIT 0 0
4 OPR 0 7
5 JPC 0 9
6 LIT 0 1
7 STO 0 3
8 JMP 0 17
9 LOD 0 4
10 LOD 0 4
11 LIT 0 1
12 OPR 0 3
13 CAL 1 1
14 RETVAL 0 1
15 OPR 0 4
16 STO 0 3
17 RET 0 0
18 INT 0 104
19 LIT 0 3
20 LIT 0 3
21 LIT 0 1
22 LIT 0 1
23 OPR 0 3
24 LIT 0 10
25 OPR 0 4
26 OPR 0 2
27 LIT 0 1
28 LIT 0 1
29 OPR 0 3
30 OPR 0 2
31 STOI 0 0
32 LIT 0 11
33 LIT 0 3
34 LIT 0 4
35 LIT 0 1
36 OPR 0 3
37 LIT 0 10
38 OPR 0 4
39 OPR 0 2
40 LIT 0 4
41 LIT 0 1
42 OPR 0 3
43 OPR 0 2
44 STOI 0 0
```

45 LIT 0 20  
46 OPR 0 1  
47 LIT 0 3  
48 LIT 0 2  
49 LIT 0 1  
50 OPR 0 3  
51 LIT 0 10  
52 OPR 0 4  
53 OPR 0 2  
54 LIT 0 3  
55 LIT 0 1  
56 OPR 0 3  
57 OPR 0 2  
58 STOI 0 0  
59 LIT 0 3  
60 LIT 0 2  
61 LIT 0 1  
62 OPR 0 3  
63 LIT 0 10  
64 OPR 0 4  
65 OPR 0 2  
66 LIT 0 3  
67 LIT 0 1  
68 OPR 0 3  
69 OPR 0 2  
70 LODI 0 0  
71 LIT 0 3  
72 LIT 0 1  
73 LIT 0 1  
74 OPR 0 3  
75 LIT 0 10  
76 OPR 0 4  
77 OPR 0 2  
78 LIT 0 1  
79 LIT 0 1  
80 OPR 0 3  
81 OPR 0 2  
82 LODI 0 0  
83 OPR 0 2  
84 LIT 0 3  
85 LIT 0 4  
86 LIT 0 1  
87 OPR 0 3  
88 LIT 0 10  
89 OPR 0 4  
90 OPR 0 2  
91 LIT 0 4  
92 LIT 0 1



```

93 OPR 0 3
94 OPR 0 2
95 LODI 0 0
96 OPR 0 2
97 LIT 0 3
98 LIT 0 2
99 LIT 0 1
100 OPR 0 3
101 LIT 0 10
102 OPR 0 4
103 OPR 0 2
104 LIT 0 3
105 LIT 0 1
106 OPR 0 3
107 OPR 0 2
108 STOI 0 0
109 LIT 0 3
110 LIT 0 2
111 LIT 0 1
112 OPR 0 3
113 LIT 0 10
114 OPR 0 4
115 OPR 0 2
116 LIT 0 3
117 LIT 0 1
118 OPR 0 3
119 OPR 0 2
120 LODI 0 0
121 OPR 0 14
122 LIT 0 5
123 CAL 1 1
124 RETVAL 0 1
125 OPR 0 14
126 RET 0 0

=== OUTPUT PROGRAM ===
-6
120

```

❖ Kasus uji 5

*Tabel 5.5 Kasus Uji 5*

| Input                    |
|--------------------------|
| program UjiKomprehensif; |

```

const
    BATAS_MIN == 1;
    BATAS_MAX == 5;
    PI == 3.14159;

type
    RentangIndeks == 1..5;
    AngkaBulat == integer;
    StatusPemain == (HIDUP, MATI, BERTAHAN, MENYERANG);
    PapanCatur == array [1..5] of array [1..5] of integer;
    Karakter == record
        id: integer;
        darah: real;
        status: integer;
        papan: array [1..5] of array [1..5] of integer;
        is_aktif: boolean
    end;

var
    hero: Karakter;
    i, j, langkah: integer;
    hasilFibo, hasilFak: integer;
    r, c: integer;

function fibonacci(n: integer): integer;
begin
    if (n <= 1) then
        fibonacci := n
    else
        fibonacci := fibonacci(n - 1) + fibonacci(n - 2);
    end;

function faktorial(n: integer): integer;
begin
    if (n == 0) then
        faktorial := 1
    else
        faktorial := n * faktorial(n - 1);
    end;

procedure inisialisasiPapan();
begin
    for r := 1 to 5 do
        begin
            for c := 1 to 5 do
                begin
                    if (r == c) then
                        hero.papan[r][c] := 1

```

```

        else if ((r + c) == 6) then
            hero.papan[r][c] := 2
        else
            hero.papan[r][c] := 0;
        end;
    end;
end;

begin
    hero.id := 99;
    hero.darah := 100.5;
    hero.status := 1;
    hero.is_aktif := true;

    inisialisasiPapan();

    hasilFibo := fibonacci(6);
    hasilFak := faktorial(4);
    writeln(hasilFibo);
    writeln(hasilFak);

    langkah := 5;
    while (langkah > 0) and (hero.is_aktif == true) do
    begin
        hero.darah := hero.darah - 10.0;
        langkah := langkah - 1;

        if (hero.darah <= 0.0) then
            hero.is_aktif := false;
        end;

        i := 1;
        repeat
            j := 1;
            repeat
                hero.papan[i][j] := (hero.papan[i][j] * 2) div 1 + (10
mod 3);
                j := j + 1;
            until j > 5;
            i := i + 1;
        until i > 5;

        langkah := 2;
        case langkah of
            1:
                hero.status := 3;
            2, 3:
                begin

```

```

        hero.status := 2;
        hero.darah := hero.darah + 25.5;
    end;
4:
    hero.status := 4
end;

if not (hero.status == 4) or (hero.darah > 50.0) then
    writeln('Hero Bertahan!')
else
    writeln('Game Over');
end.

```

### Output

```

=== INTERMEDIATE CODE ===
0 JMP 0 78
1 INT 0 4
2 LIT 0 1
3 STO 1 37
4 LOD 1 37
5 LIT 0 5
6 OPR 0 12
7 JPC 0 39
8 LIT 0 1
9 STO 1 38
10 LOD 1 38
11 LIT 0 5
12 OPR 0 12
13 JPC 0 34
14 LOD 1 37
15 LOD 1 38
16 OPR 0 7
17 JPC 0 20
18 LIT 0 1
19 JMP 0 29
20 LOD 1 37
21 LOD 1 38
22 OPR 0 2
23 LIT 0 6
24 OPR 0 7
25 JPC 0 28
26 LIT 0 2
27 JMP 0 29
28 LIT 0 0
29 LOD 1 38
30 LIT 0 1

```

```
31 OPR 0 2
32 STO 1 38
33 JMP 0 10
34 LOD 1 37
35 LIT 0 1
36 OPR 0 2
37 STO 1 37
38 JMP 0 4
39 RET 0 0
40 INT 0 5
41 LOD 0 4
42 LIT 0 1
43 OPR 0 12
44 JPC 0 48
45 LOD 0 4
46 STO 0 3
47 JMP 0 60
48 LOD 0 4
49 LIT 0 1
50 OPR 0 3
51 CAL 1 40
52 RETVAL 0 1
53 LOD 0 4
54 LIT 0 2
55 OPR 0 3
56 CAL 1 40
57 RETVAL 0 1
58 OPR 0 2
59 STO 0 3
60 RET 0 0
61 INT 0 5
62 LOD 0 4
63 LIT 0 0
64 OPR 0 7
65 JPC 0 69
66 LIT 0 1
67 STO 0 3
68 JMP 0 77
69 LOD 0 4
70 LOD 0 4
71 LIT 0 1
72 OPR 0 3
73 CAL 1 61
74 RETVAL 0 1
75 OPR 0 4
76 STO 0 3
77 RET 0 0
78 INT 0 39
```

```
79 LIT 0 99
80 STO 0 3
81 LITR 0 0
82 STO 0 4
83 LIT 0 1
84 STO 0 5
85 LIT 0 1
86 STO 0 31
87 CAL 0 1
88 LIT 0 6
89 CAL 1 40
90 RETVAL 0 1
91 STO 0 35
92 LIT 0 4
93 CAL 1 61
94 RETVAL 0 1
95 STO 0 36
96 LOD 0 35
97 OPR 0 14
98 LOD 0 36
99 OPR 0 14
100 LIT 0 5
101 STO 0 34
102 LOD 0 34
103 LIT 0 0
104 OPR 0 11
105 JPC 0 112
106 LOD 0 31
107 LIT 0 1
108 OPR 0 7
109 JPC 0 112
110 LIT 0 1
111 JMP 0 113
112 LIT 0 0
113 JPC 0 129
114 LOD 0 4
115 LITR 0 1
116 OPR 0 3
117 STO 0 4
118 LOD 0 34
119 LIT 0 1
120 OPR 0 3
121 STO 0 34
122 LOD 0 4
123 LITR 0 2
124 OPR 0 12
125 JPC 0 128
126 LIT 0 0
```

```
127 STO 0 31
128 JMP 0 102
129 LIT 0 1
130 STO 0 32
131 LIT 0 1
132 STO 0 33
133 LIT 0 0
134 LIT 0 2
135 OPR 0 4
136 LIT 0 1
137 OPR 0 5
138 LIT 0 10
139 LIT 0 3
140 OPR 0 6
141 OPR 0 2
142 LOD 0 33
143 LIT 0 1
144 OPR 0 2
145 STO 0 33
146 LOD 0 33
147 LIT 0 5
148 OPR 0 11
149 LIT 0 0
150 OPR 0 7
151 JPC 0 133
152 LOD 0 32
153 LIT 0 1
154 OPR 0 2
155 STO 0 32
156 LOD 0 32
157 LIT 0 5
158 OPR 0 11
159 LIT 0 0
160 OPR 0 7
161 JPC 0 131
162 LIT 0 2
163 STO 0 34
164 LOD 0 34
165 LIT 0 1
166 OPR 0 7
167 JPC 0 171
168 LIT 0 3
169 STO 0 5
170 JMP 0 193
171 LOD 0 34
172 LIT 0 2
173 OPR 0 7
174 JPC 0 186
```

```
175 LOD 0 34
176 LIT 0 3
177 OPR 0 7
178 JPC 0 186
179 LIT 0 2
180 STO 0 5
181 LOD 0 4
182 LITR 0 3
183 OPR 0 2
184 STO 0 4
185 JMP 0 193
186 LOD 0 34
187 LIT 0 4
188 OPR 0 7
189 JPC 0 193
190 LIT 0 4
191 STO 0 5
192 JMP 0 193
193 LOD 0 5
194 LIT 0 4
195 OPR 0 7
196 LIT 0 0
197 OPR 0 7
198 JPC 0 201
199 LIT 0 1
200 JMP 0 208
201 LOD 0 4
202 LITR 0 4
203 OPR 0 11
204 JPC 0 207
205 LIT 0 1
206 JMP 0 208
207 LIT 0 0
208 JPC 0 212
209 LITS 0 0
210 OPR 0 14
211 JMP 0 214
212 LITS 0 1
213 OPR 0 14
214 RET 0 0

=== OUTPUT PROGRAM ===
8
24
Hero Bertahan!
```



## **BAB 6**

### **KESIMPULAN DAN SARAN**

#### **6.1. Kesimpulan**

Pada milestone ini telah berhasil diimplementasikan tahap keempat dari pipeline compiler untuk bahasa Arion, yaitu Intermediate Code Generation (ICG) dan eksekusi bytecode oleh Interpreter. Proses ini terdiri dari dua tahap utama: Pembangkitan bytecode dari Decorated AST menggunakan generator rekursif top-down dengan pola `genXxx()` per jenis node AST, yang menghasilkan instruksi dalam satu lintasan menggunakan teknik backpatching, dan Eksekusi bytecode oleh mesin virtual berbasis stack melalui fetch-decode-execute loop yang mengelola stack frame, program counter, dan base pointer secara eksplisit.

Instruksi set yang digunakan terdiri dari empat belas opcode, yaitu LIT, LITS, LITR, LOD, STO, CAL, INT, JMP, JPC, OPR, RET, RETVAL, LODI, dan STOI, untuk mengekspresikan seluruh konstruksi bahasa Arion yang mencakup struktur kontrol bersarang, pemanggilan fungsi dan prosedur, akses array multidimensi, akses field record, serta operasi aritmatika campuran integer dan real. Tata letak stack frame menggunakan tiga slot overhead (static link, dynamic link, return address) diikuti satu slot return value, lalu argumen dan variabel lokal, sehingga alamat seluruh variabel dapat ditentukan secara statis pada waktu kompilasi tanpa memerlukan informasi runtime tambahan.

#### **6.2. Saran**

Untuk pengembangan selanjutnya, terdapat beberapa hal yang dapat ditingkatkan. Pertama, dukungan instruksi `readln` saat ini belum diimplementasikan sepenuhnya dan hanya menghasilkan peringatan di console, sehingga perlu diselesaikan dengan menambahkan opcode atau OPR code baru yang membaca nilai dari standard input dan menyimpannya langsung ke alamat variabel yang dituju. Kedua, mekanisme pass-by-reference belum didukung karena tata letak frame saat ini hanya meneruskan nilai argumen; untuk mendukung parameter var seperti pada Pascal standar, perlu ditambahkan penerusan alamat secara eksplisit dengan memanfaatkan instruksi LODI/STOI pada sisi callee. Ketiga, sistem tagged integer untuk nilai real dan string memiliki keterbatasan kapasitas karena jumlah nilai yang dapat diinternkan dibatasi oleh panjang bit mask yang tersisa, sehingga untuk program skala besar representasi ini perlu diganti dengan manajemen tabel yang lebih fleksibel. Keempat, belum ada optimasi kode pada tahap ICG sehingga dapat ditambahkan optimasi sederhana seperti constant folding dan dead code elimination untuk meningkatkan efisiensi eksekusi.

## LAMPIRAN

**Repository GitHub:** <https://github.com/SyaqinaOctavia/MMD-Tubes-IF2224-2026.git>

| NIM      | Kontribusi Tugas                                                                                                                     | Persentase |
|----------|--------------------------------------------------------------------------------------------------------------------------------------|------------|
| 13524040 | Testing program, Testcase input-output, writeln method, array access fixing, ICG executors fixing, Interpreter fixing, parser fixing | 25%        |
| 13524042 | Metode dalam ICG, class diagram, Penulisan laporan                                                                                   | 25%        |
| 13524048 | Metode dalam ICG, setup header ICG dan interpreter, Debugging interpreter                                                            | 25%        |
| 13524088 | Metode dalam ICG, Penulisan laporan                                                                                                  | 25%        |

## REFERENSI

John E., Rajeev M., dan Jeffrey D. *Introduction to Automata Theory, Languages, and Computation 3rd Edition*.  
<https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613-Introduction-to-Automata-Theory,-Languages,-and-Computation-Edit ion-3.pdf>

GeeksForGeeks. *Intermediate Code Generation in Compiler Design*. Diakses 6 Juni 2026 dari  
<https://www.geeksforgeeks.org/compiler-design/intermediate-code-generation-in-compiler-design/>

Philip Koopman. *Generic Stack Machine*. Diakses 6 Juni 2026 dari  
[https://users.ece.cmu.edu/~koopman/stack\\_computers/sec3\\_2.html](https://users.ece.cmu.edu/~koopman/stack_computers/sec3_2.html)